Arquitetura: Sistema Autônomo de Geração de Conteúdo Viral

Autor: Moa · **Data:** 04/11/2026 · **Status:** Rascunho v1 (fase de arquitetura)

1. Visão geral

Pipeline autônomo que roda todo dia de madrugada. Ele descobre vídeos virais no TikTok dentro de critérios definidos pelo dono, extrai a estrutura e o tema do melhor candidato, escreve um roteiro **original** com gancho forte nos três primeiros segundos, gera narração com a voz clonada do próprio criador, monta um vídeo com avatar virtual em formato reaction (criador pequeno na tela) e agenda a publicação em TikTok, Instagram Reels e YouTube Shorts. Meta inicial: 6 vídeos por dia.

Princípio de design: cada etapa é um módulo independente, com entrada e saída bem definidas, para poder trocar ferramentas sem reescrever o sistema.

2. Requisitos

2.1 Requisitos funcionais (obrigatórios)

- RF01: Executar automaticamente todos os dias às 01:00 (fuso America/Sao_Paulo).
- RF02: Coletar os 30 vídeos mais relevantes do TikTok por nicho/hashtag/palavra-chave configurável.
- RF03: Calcular um score de viralidade configurável (views, likes, comentários, compartilhamentos, idade do vídeo, velocidade de crescimento).
- RF04: Selecionar o melhor vídeo por score, ignorando os já processados.
- RF05: Baixar o vídeo selecionado e extrair o áudio.
- RF06: Transcrever o áudio com marcação de tempo.
- RF07: Gerar um roteiro original (tema inspirado, texto novo) com gancho nos primeiros 3 segundos, desenvolvimento e CTA.
- RF08: Gerar narração com a voz clonada do criador.
- RF09: Gerar o vídeo final com avatar do criador, legendas e o conteúdo de referência, em formato vertical 9:16.
- RF10: Agendar e publicar no TikTok, Instagram Reels e YouTube Shorts com título, descrição e hashtags.
- RF11: Registrar tudo em banco de dados (vídeo de origem, roteiro, arquivos gerados, status de cada etapa, links publicados).

- RF12: Produzir 6 vídeos por dia (6 execuções do fluxo ou 6 candidatos distintos).

2.2 Requisitos não funcionais

- RNF01: Idempotência. Reexecutar uma etapa não duplica vídeo nem publicação.
- RNF02: Tolerância a falhas. Retry com backoff e falha de uma etapa não derruba o dia inteiro.
- RNF03: Observabilidade. Logs estruturados e alerta no Telegram em caso de erro ou sucesso.
- RNF04: Configuração externa (arquivo YAML/env), sem valores fixos no código.
- RNF05: Segredos (tokens, chaves) fora do repositório.
- RNF06: Custo controlado: priorizar ferramentas gratuitas ou open source.
- RNF07: Modularidade: cada etapa substituível.

2.3 Requisitos desejáveis (não obrigatórios)

- RD01: Aprovação humana opcional via Telegram antes de publicar (modo supervisionado).
- RD02: Painel web simples com fila, histórico e métricas.
- RD03: Coleta de métricas pós-publicação para retroalimentar o score de viralidade.
- RD04: A/B test de ganchos.
- RD05: Suporte a múltiplos nichos e múltiplas contas.
- RD06: Geração de thumbnail e capa.

3. Riscos e restrições

- **Direitos autorais:** baixar o vídeo é só para análise. O roteiro precisa ser transformado de verdade, e o vídeo final não deve reutilizar o material original sem transformação clara. Mesmo assim, é uma zona cinza; vale manter a abordagem de inspiração de estrutura.
- **Termos de uso e automação:** TikTok e Instagram limitam scraping e publicação automatizada. Risco de bloqueio de conta. Mitigar usando APIs oficiais quando existirem, ritmo humano de postagem e conta de teste no início.
- **Conteúdo repetitivo e em massa:** as plataformas penalizam spam e conteúdo reciclado. Seis vídeos por dia exigem variedade real.
- **Identidade sintética:** voz e avatar só do próprio criador, e vale avaliar o rótulo de conteúdo gerado por IA que as plataformas exigem.
- **Fragilidade da coleta:** scrapers quebram quando o TikTok muda. Isolar essa etapa atrás de uma interface.

4. Arquitetura

4.1 Visão em camadas

1. **Orquestração:** agendador (cron ou APScheduler) dispara o fluxo diário e controla a fila de jobs.
2. **Coleta (Discovery):** busca e ranqueia vídeos.
3. **Processamento (Analysis):** download, extração de áudio, transcrição e geração de roteiro.
4. **Produção (Studio):** voz, avatar, montagem e legendas.
5. **Publicação (Publisher):** agendamento e envio para as três plataformas.
6. **Persistência e observabilidade:** banco de dados, armazenamento de arquivos, logs e notificações.

4.2 Fluxo (pipeline)

```
flowchart TD
  A[Cron 01:00] --> B[Discovery: buscar 30 vídeos no TikTok]
  B --> C[Scoring: calcular viralidade]
  C --> D{Já processado?}
  D -- Sim --> C
  D -- Não --> E[Download do vídeo]
  E --> F[Extrair áudio]
  F --> G[Transcrição com Whisper]
  G --> H[LLM: análise de estrutura + roteiro original]
  H --> I[Voz clonada: TTS]
  I --> J[Avatar + montagem FFmpeg + legendas]
  J --> K{Aprovação humana ativa?}
  K -- Sim --> L[Telegram: aprovar ou rejeitar]
  K -- Não --> M[Agendar publicação]
  L --> M
  M --> N[TikTok]
  M --> O[Instagram Reels]
  M --> P[YouTube Shorts]
  N --> Q[Registrar no banco + notificar]
  O --> Q
  P --> Q
```

4.3 Módulos e responsabilidades

Módulo	Entrada	Saída	Opções de ferramenta
Scheduler	horário	disparo do job	cron, APScheduler, Celery Beat
Discovery	nicho, hashtags, parâmetros	lista de candidatos	TikTok Research API, Apify, yt-dlp + scraper próprio
Scoring	candidatos	candidato vencedor	Python puro, fórmula configurável
Downloader	URL	arquivo mp4	yt-dlp
Transcriber	áudio	texto com timestamps	Whisper (local ou faster-whisper)
Scriptwriter	transcrição + persona	roteiro original estruturado	API Claude
Voice	roteiro	arquivo de áudio	Voice Studio (open source), XTTS, ElevenLabs
Avatar	foto/vídeo base + áudio	vídeo do avatar	SadTalker, Wav2Lip, HeyGen API
Composer	avatar + reference + legendas	mp4 final 9:16	FFmpeg, MoviePy
Publisher	mp4 + metadados	links publicados	APIs oficiais ou ferramenta de agendamento
Notifier	eventos	mensagens	Telegram Bot

4.4 Modelo de dados (rascunho)

- **source_videos:** id, plataforma, url, autor, métricas, score, coletado_em, status.
- **scripts:** id, source_video_id, transcrição, roteiro, gancho, versão.
- **assets:** id, script_id, tipo (áudio, avatar, final), caminho, duração.
- **publications:** id, asset_id, plataforma, agendado_para, publicado_em, url, status.
- **runs:** id, data, etapa, status, erro, duração.

4.5 Stack sugerida

- **Linguagem:** Python 3.12 (ecossistema de vídeo e IA).
- **Orquestração de jobs:** Celery + Redis, ou fila simples com APScheduler na primeira versão.

- **Banco:** PostgreSQL.
- **Armazenamento:** disco local na VPS, com opção de S3/R2 depois.
- **Infra:** VPS com Docker Compose. Etapas pesadas (avatar, Whisper) pedem GPU ou tempo de processamento maior.
- **Observabilidade:** logs estruturados + Telegram.

5. Decisões a tomar (em aberto)

- Como coletar do TikTok: API oficial de pesquisa, serviço terceiro ou scraper próprio?
- Avatar: geração local (exige GPU) ou API paga?
- Publicação: APIs oficiais de cada plataforma ou ferramenta de agendamento intermediária?
- Quais exatamente são os parâmetros de viralidade e o nicho do canal?
- Modo supervisionado (aprovação humana) no início: sim ou não?

6. Roadmap

Fase 0: Fundação (semana 1)

Definir nicho e parâmetros de viralidade, fechar as decisões em aberto, montar repositório, Docker Compose, banco e configuração. Entrega: projeto rodando vazio com logs e notificação no Telegram.

Fase 1: Discovery e Scoring (semana 2)

Coletar vídeos, calcular score, guardar no banco. Entrega: comando que lista o top 30 e escolhe o vencedor.

Fase 2: Análise e roteiro (semana 3)

Download, transcrição e geração de roteiro original com gancho. Entrega: roteiro de qualidade a partir de um link, validado manualmente.

Fase 3: Produção (semanas 4 e 5)

Voz clonada, avatar, montagem e legendas. Entrega: mp4 final 9:16 pronto para postar, gerado ponta a ponta.

Fase 4: Publicação (semana 6)

Integração com as três plataformas, agendamento e registro. Entrega: um vídeo publicado automaticamente em conta de teste.

Fase 5: Automação e robustez (semana 7)

Cron diário, retries, idempotência, alertas, modo supervisionado. Entrega: sistema rodando sozinho por 7 dias seguidos.

Fase 6: Escala e melhoria (semana 8 em diante)

Subir para 6 vídeos por dia, métricas pós-publicação, A/B de ganchos, painel web.

7. Próximos passos imediatos

1. Responder as decisões em aberto da seção 5.
2. Validar o documento e ajustar requisitos.
3. Começar a Fase 0 e só então partir para o código.